

Top 30 Core Java Interview Questions

Commonly asked in QA Automation interviews - with concise model answers

HOW TO USE THIS GUIDE

Answer each question aloud first. Then compare your response with the model answer and add one example from your own Selenium, API, or framework experience.

Topics covered: OOP, strings, collections, exceptions, generics, streams, concurrency, and framework configuration.

1 What are the main OOP principles in Java?

Model answer: Encapsulation hides state behind methods; inheritance reuses or specializes behavior; polymorphism lets one interface support many implementations; abstraction exposes essential behavior while hiding details. In automation, Page Objects combine all four principles.

2 What is the difference between a class and an object?

Model answer: A class is a blueprint defining fields and methods. An object is a runtime instance of that class with its own state. Example: LoginPage is a class; the page object created for a browser session is an object.

3 What is the difference between method overloading and overriding?

Model answer: Overloading uses the same method name with different parameter lists and is resolved at compile time. Overriding replaces an inherited method with the same signature and is resolved at runtime. Return type alone cannot overload a method.

4 Interface vs abstract class - when would you use each?

Model answer: An interface defines a contract and supports multiple inheritance of type. An abstract class can hold shared state, constructors, and implemented methods. Use interfaces for capabilities such as Screenshotable; use an abstract base page for shared driver and wait behavior.

5 What do public, protected, default, and private mean?

Model answer: public is accessible everywhere; protected within the package and by subclasses; default (no modifier) within the package; private only inside the declaring class. Prefer the narrowest access that supports the design.

6 What is the difference between static and instance members?

Model answer: A static member belongs to the class and is shared; an instance member belongs to each object. Static methods cannot directly access instance state. Avoid sharing mutable static drivers in parallel test execution.

7 What are final, finally, and finalize?

Model answer: final prevents reassignment, overriding, or inheritance depending on context. finally is a cleanup block after try/catch. finalize() was an unreliable garbage-collection hook and is deprecated; use try-with-resources for deterministic cleanup.

8 Why is String immutable in Java?

Model answer: After creation, a String value cannot change. Immutability enables safe sharing, string pooling, stable hash codes, and better security. Operations such as concat create a new String rather than modifying the original.

9 What is the difference between == and equals()?

Model answer: == compares primitive values or object references. equals() compares logical content when a class overrides it. For strings and collection values, normally use equals(); use Objects.equals(a, b) when null safety is useful.

10 String vs StringBuilder vs StringBuffer?

Model answer: String is immutable. StringBuilder is mutable and fastest for ordinary single-threaded concatenation. StringBuffer is mutable and synchronized, so it has extra overhead. Use StringBuilder when assembling logs, payloads, or dynamic test data.

11 How does the Java Collections Framework help automation?

Model answer: List stores ordered values and allows duplicates; Set stores unique values; Map stores key-value pairs; Queue models ordered processing. Automation commonly uses List for elements, Set for window handles, and Map for test data or headers.

12 ArrayList vs LinkedList?

Model answer: ArrayList uses a resizable array and is generally better for indexed access and iteration. LinkedList is node-based and can add/remove efficiently at known ends or positions, but has slower random access and more memory overhead. ArrayList is the usual default.

13 HashSet vs LinkedHashSet vs TreeSet?

Model answer: HashSet gives no iteration-order guarantee; LinkedHashSet preserves insertion order; TreeSet keeps elements sorted and requires comparable elements or a Comparator. All enforce uniqueness.

14 How does HashMap work, and can it contain nulls?

Model answer: HashMap uses hashCode() to select a bucket and equals() to identify the matching key. It allows one null key and multiple null values. Average get/put is constant time with a good hash distribution; it is not thread-safe.

15 Comparable vs Comparator?

Model answer: Comparable defines a class's natural order through compareTo(). Comparator defines an external or alternative order through compare(). Comparator is useful when test results must be sorted by duration, status, or name without changing the model class.

16 Why must equals() and hashCode() be consistent?

Model answer: Equal objects must return the same hash code. Hash-based collections first use hashCode() and then equals(); violating the contract can make logically equal keys appear missing or duplicated.

17 Checked vs unchecked exceptions?

Model answer: Checked exceptions must be caught or declared and usually represent recoverable external conditions. Unchecked exceptions extend RuntimeException and often signal programming or validation errors. Automation frameworks should add context, not hide failures with broad catch blocks.

18 What is the purpose of try, catch, finally, throw, and throws?

Model answer: try encloses risky code; catch handles a matching exception; finally performs cleanup; throw creates or propagates a specific exception; throws declares possible exceptions in a method signature.

19 What is try-with-resources?

Model answer: It automatically closes AutoCloseable resources, even when an exception occurs. Use it for files, streams, database connections, and similar resources. It is safer and shorter than manual cleanup in finally.

20 How would you create a custom exception for a framework?

Model answer: Extend Exception for a checked type or RuntimeException for an unchecked type, add useful constructors, and preserve the original cause. Example: `throw new TestDataException("Cannot load users.json", cause);`

21 What is the difference between an array and a collection?

Model answer: An array has fixed length and can store primitives or objects. Collections resize dynamically, store objects, and provide richer APIs. Arrays are suitable for fixed data; collections are usually more flexible for dynamic test results.

22 What are Java generics and why are they useful?

Model answer: Generics provide compile-time type safety and reduce casting. List prevents unrelated objects from being inserted. Generic helpers such as `T read(String key, Class type)` can support reusable, typed framework utilities.

23 What are lambda expressions and functional interfaces?

Model answer: A lambda is a concise implementation of a functional interface, which has one abstract method. Examples include Predicate, Function, Consumer, and Runnable. They are useful for filters, waits, callbacks, and data transformations.

24 What is the Stream API?

Model answer: Streams process data declaratively through operations such as filter, map, sorted, and collect. Intermediate operations are lazy; a terminal operation triggers execution. Streams do not store data and should not be confused with I/O streams.

25 What is the difference between map() and flatMap()?

Model answer: map transforms each input into one output. flatMap transforms each input into a stream-like result and flattens those results into one stream. It is useful when each test suite contains many tests and one combined list is needed.

26 How should Optional be used?

Model answer: Optional models a possibly absent return value and supports methods such as map, orElse, and orElseThrow. It is most useful as a return type. Avoid calling get() without checking and avoid using Optional for every field or parameter.

27 What is multithreading, and what problems arise in parallel tests?

Model answer: Multithreading runs multiple execution paths within a process. Shared mutable state can cause race conditions and flaky tests. Isolate drivers and test data per thread, avoid unsafe statics, and synchronize only genuinely shared resources.

28 synchronized vs volatile?

Model answer: synchronized provides mutual exclusion and visibility around a block or method. volatile guarantees visibility of the latest value but does not make compound actions such as count++ atomic. Use concurrent utilities when they fit the problem.

29 What is ThreadLocal and why is it used in Selenium frameworks?

Model answer: ThreadLocal stores a separate value for each thread. A parallel framework may keep one WebDriver per test thread. Always remove the value during cleanup to reduce leakage in reused thread pools.

30 How do you read configuration safely in Java automation?

Model answer: Use Properties or a typed configuration layer, load once through try-with-resources, validate required keys, and allow environment-specific overrides. Do not hard-code credentials or commit secrets; obtain them from environment variables or a secret manager.